

Individual Report — Student 4

Ablation Studies, Visualization, and Theoretical Analysis for the Dual-Arm Diffusion Policy Project

Student 4
City University of Hong Kong

1 Role and Contributions

In this team project, I served as Student 4, responsible for the **full experiment pipeline, visualization, statistical analysis, final report writing, and theoretical discussion**. My specific deliverables include:

1. **Ablation Experiments:** Designed and executed 12 independent training runs of the Diffusion Policy under three controlled axes (demonstration quantity, prediction horizon, and observation horizon), totaling approximately 1.5 hours of GPU training on an NVIDIA RTX PRO 6000 (96GB).
2. **Visualization Pipeline:** Implemented a unified matplotlib-based pipeline that renders all 12 figures in the team report, including success-rate comparisons, training-loss curves, t-SNE state distributions, and the radar plot summarizing cross-method trade-offs.
3. **Statistical Analysis:** Aggregated raw outputs from Students 1–3 (BC test MSE, DAgger iteration logs, Diffusion Policy loss curves) into the final result tables and quantified covariate shift via Maximum Mean Discrepancy (MMD).
4. **Final Report and Theoretical Discussion:** Authored the team’s 12-page LaTeX report, including the theoretical treatment of covariate shift, the sim-to-real discussion, and the integration of all sub-team results into a coherent narrative.

The work of other team members provided the inputs to my pipeline: Student 1 produced the dual-arm Robosuite environment and 500 expert demonstrations; Student 2 implemented BC and DAgger; Student 3 wrote the Diffusion Policy training code (`ConditionalUnet1D`). My ablation code reuses Student 3’s model architecture without modification, wrapping it in an automated experiment-management layer.

2 Ablation Studies

2.1 Motivation and Design

Beyond reporting an aggregate success rate, the project specification requires a controlled study of which design choices actually matter for diffusion-based imitation learning. I therefore selected three hyperparameters that span data, output structure, and input structure:

- **Demonstration count** ($N \in \{50, 100, 200, 300, 500\}$): tests how data scale drives performance.
- **Prediction horizon** ($H_p \in \{4, 8, 16, 32\}$): tests how the length of the action chunk produced by the model affects learning.
- **Observation horizon** ($H_o \in \{1, 2, 4\}$): tests how much past context the model needs.

For each axis, only one variable changes at a time; all other hyperparameters are fixed at the team default ($H_p = 16$, $H_o = 2$, $N = 500$). All runs use **100 epochs** and **batch size 256**; this is enough to observe converged trends while keeping the total budget manageable, since I run 12 experiments back-to-back. The model, optimizer (AdamW, $\text{lr} = 10^{-4}$, $\text{wd} = 10^{-6}$), EMA (decay 0.999), DDPM scheduler (100 steps, squared-cosine), and cosine LR schedule are identical to the team default in Student 3’s training code.

2.2 Demonstration Quantity

Demos	Samples	Final Loss	Time (min)
50	5,708	0.01875	1.2
100	11,394	0.01053	2.0
200	22,789	0.00625	3.9
300	34,199	0.00393	5.8
500	57,075	0.00220	9.6

Table 1: Ablation on number of expert demonstrations.

The dominant trend (Table 1) is a near-linear decrease in log-loss as demonstrations grow geometrically: each doubling of N reduces the final loss by roughly 40–45%. This is the strongest single effect observed in any of my ablations. Importantly, the curve has not flattened at $N = 500$, indicating that the model is not yet data-saturated for this task complexity.

Practical implication. For diffusion-based imitation learning on coordinated dual-arm manipulation, *collecting more demonstrations gives consistently larger returns than tuning the model architecture*. This finding guides the team’s recommendation that future work should prioritize expanding the demonstration set rather than searching over model hyperparameters.

2.3 Prediction Horizon (Action Chunking)

Pred Horizon	Samples	Final Loss	Time (min)
4	63,075	0.00207	10.6
8	61,075	0.00251	10.3
16	57,075	0.00224	9.7
32	49,075	0.00229	8.5

Table 2: Ablation on prediction horizon (action chunk length).

Final losses lie within a narrow band of 0.00207–0.00251 (Table 2), even when the chunk length varies by $8\times$. Two design choices explain this robustness:

1. **Receding-horizon execution.** Although the model predicts H_p steps, only the first $H_a = 2$ steps are executed before re-planning. The remaining steps act as a soft temporal prior, not a hard commitment, so high accuracy is required only over a short prefix.
2. **Multi-scale 1D U-Net.** Longer sequences are absorbed by additional downsampling rather than by more learnable parameters per timestep, so the per-step prediction difficulty is roughly invariant to H_p .

Caveat. Training loss alone does not capture downstream benefits of longer chunks (e.g., trajectory smoothness in closed-loop execution). A definitive ablation would require online evaluation, which we deferred due to environment access constraints.

2.4 Observation Horizon (History Length)

Obs Horizon	Cond Dim	Final Loss	Time (min)
1	12	0.00230	9.7
2	24	0.00229	9.6
4	48	0.00227	9.5

Table 3: Ablation on observation horizon (history length).

The three configurations differ by less than 1.5% in final loss (Table 3). Physically, the pick-and-place task is quasi-static at 20 Hz: the cube does not move on its own, the end-effectors move at ≤ 1 cm/step, and the goal is fixed. A single observation frame already encodes the full Markov state. Velocity or acceleration cues, which would require multiple frames, contribute negligible information here.

When this finding would not generalize. For tasks with fast-moving objects (e.g., catching, hitting) or human-in-the-loop intent estimation, multi-frame history is expected to matter much more, since velocity is no longer derivable from a single frame.

2.5 Cross-Ablation Summary

Synthesizing the three studies:

- **Data $\gg$ architecture.** The variance in final loss induced by the demo-count axis ($\approx 8.5\times$ between $N = 50$ and $N = 500$) dwarfs the variance from either horizon axis ($\leq 1.2\times$).
- **The default configuration is well-chosen.** The team’s default ($H_p = 16$, $H_o = 2$) sits near the minimum of both horizon ablations, indicating that further hyperparameter search on these axes would yield diminishing returns.
- **The bottleneck is data, not capacity.** Since the model has not saturated at 500 demos, the practical next step is to enlarge the demonstration set, not to enlarge the network.

3 Visualization and Statistical Analysis

3.1 Unified Plotting Pipeline

The script `student4_visualizations.py` centralizes the generation of all figures used in the team report. It enforces a common visual style (color palette, font size, DPI, grid alpha) so that figures sourced from disparate experiments remain visually coherent. The pipeline produces:

- Aggregate comparisons: success-rate bar chart (Fig. 1), method radar plot (Fig. 12), summary table image (Fig. 7).
- Training dynamics: BC training curve (Fig. 2), Diffusion training curve (Fig. 4), BC vs. Diffusion log-scale comparison (Fig. 5), four-phase Diffusion convergence breakdown (Fig. 9).
- Method-specific diagnostics: DAgger iteration improvement (Fig. 3), average episode length (Fig. 8).

- Conceptual figures: covariate-shift illustration (Fig. 10), action-chunking schematic (Fig. 11).
- Distribution analysis: t-SNE of states visited by Expert, BC, and DAgger policies (Fig. 6).
- Ablation outputs: training-loss curves and bar charts for all three ablation axes, plus a summary table image.

3.2 Quantifying Covariate Shift

To complement the qualitative t-SNE visualization, I computed the Maximum Mean Discrepancy (MMD) between the state distributions of the expert policy and each learned policy:

$$\text{MMD}^2(P, Q) = \mathbb{E}[k(x, x')] - 2\mathbb{E}[k(x, y)] + \mathbb{E}[k(y, y')] \quad (1)$$

with an RBF kernel. The expert–BC MMD is substantially larger than the expert–DAgger MMD, providing a quantitative confirmation of what the t-SNE projection shows visually: BC visits states the expert never reached, while DAgger’s iterative on-policy correction pulls its visitation distribution back toward the expert’s.

4 Theoretical Discussion

4.1 Why BC Fails: A Compounding-Error View

For a deterministic learned policy π_θ with single-step error ϵ measured under the expert distribution, the worst-case performance gap relative to the expert satisfies

$$J(\pi_\theta) - J(\pi^*) \leq T \cdot \epsilon + T^2 \cdot \epsilon^2, \quad (2)$$

where T is the task horizon. The quadratic term is the formal statement of compounding error: at each step, the agent has some probability of leaving the expert’s support, and once it does, the training signal provides no guidance. For our task ($T = 500$ steps), even an offline MSE as small as 0.0058 translates to a 56-percentage-point gap in success rate (100% vs. 44%).

4.2 Why DAgger Helps

DAgger replaces the worst-case quadratic dependence with an expectation under π_θ ’s induced distribution, recovering an $O(T\epsilon)$ regret bound. Empirically, our DAgger run rises from 44% to 86% over its iterative training, with a non-monotonic curve (drop at iteration 3, recovery thereafter). The non-monotonicity is a known artifact of distribution shift in the aggregated dataset: a temporarily poor policy contributes low-quality on-policy samples that briefly degrade learning before later iterations recover.

4.3 Why Diffusion Policy Is Different

The Diffusion Policy attacks the same problem from a different angle: rather than correcting the training distribution online, it models a *conditional distribution* $p(\mathbf{A}_t \mid \mathbf{O}_t)$ over action sequences. Two consequences follow.

Multi-modality. For states where multiple expert actions are valid (e.g., either arm may grasp first), a deterministic regressor like BC averages the modes and produces a physically inconsistent action. Diffusion Policy samples one mode coherently.

Temporal commitment. Because each denoising pass produces an entire H_p -step chunk, the sampled action sequence is internally consistent. Combined with receding-horizon execution, this provides robustness to single-step prediction noise that BC and DAgger cannot match.

In our experiments, the Diffusion Policy achieves a final training loss of 4.4×10^{-4} , an order of magnitude lower than BC’s 5.8×10^{-3} . We estimate its online success rate at $\sim 90\%$; this estimate is necessarily imprecise, since training loss is only a proxy for closed-loop performance, but the gap to BC’s loss strongly suggests Diffusion Policy would surpass DAgger if both were evaluated under identical conditions.

5 Sim-to-Real Considerations

The three methods carry different deployment implications:

- **BC** is the cheapest to deploy but least robust to domain gaps. Any sim-to-real mismatch compounds through covariate shift.
- **DAgger** requires expert access during training. In simulation this is trivial (the scripted policy is always available); in the real world it usually means human teleoperation, which is expensive and slow. Variants such as HG-DAgger reduce the cost via intermittent corrections.
- **Diffusion Policy** is the most attractive for real-robot deployment: it trains entirely offline, naturally handles the multi-modal action distributions induced by domain randomization, and benefits from temporal robustness via action chunking. The main concern is inference latency from the 100-step denoising process; recent work on DDIM acceleration and consistency-distilled diffusion is directly applicable.

6 Limitations of My Contribution

1. **Training-loss-only ablation.** All ablation conclusions rest on training loss, not online success rate. Although loss is a strong correlate, online evaluation would be required for a definitive ranking, especially on the prediction- horizon axis where temporal smoothness is not captured by loss.
2. **Single seed per configuration.** Each ablation cell is a single training run. The narrow gaps in the horizon ablations ($\leq 1.5\%$) are within plausible seed-to-seed variance; replicate runs would tighten the conclusions.
3. **Limited axes.** I varied three hyperparameters; the project specification also lists noise level as a candidate axis, which I did not explore.
4. **Aggregate visualization, not raw re-evaluation.** Several figures (e.g., success-rate comparison) consume numbers produced by other team members. When their numbers update (as happened with the corrected DAgger result), the figures must be regenerated.

7 Conclusion

My contribution to this project breaks down into four interlocking deliverables: a controlled ablation study isolating the effect of data quantity, prediction horizon, and observation horizon on Diffusion Policy training; a unified visualization pipeline producing all 12 figures used in the team report; a statistical synthesis of the team’s results into the main results table and the MMD quantification of covariate shift; and the writing of the team’s final report including its theoretical and sim-to-real discussion.

The single most important quantitative finding from my ablations is that **data quantity dominates hyperparameter choice** for this task: an $8.5\times$ reduction in final loss from scaling demos $50 \rightarrow 500$, against $\leq 1.2\times$ variation across either horizon axis. This finding is operationally useful: it tells the team where to invest effort in any follow-up work.

The single most important qualitative observation is that the three methods do not occupy a single performance axis but instead trade off along several dimensions — success rate, training cost, sample efficiency, multi-modal capability, and real-world deployability — which the team’s radar plot makes explicit.

Appendix: My Submission Layout

```
Student4_Submission/
  ablation_code/
    run_ablation.py          # 12-experiment driver (mine)
    model.py                 # ConditionalUnet1D (Student 3, unchanged)
  ablation_results/
    all_results.json         # final losses + configs
    demos_{50,100,200,300,500}/
    pred_horizon_{4,8,16,32}/
    obs_horizon_{1,2,4}/
    ablation_*.png           # rendered comparison plots
  visualization_code/
    student4_visualizations.py
  figures/                   # all 12 figures (PNG)
  figures_pdf/               # all figures (PDF, for LaTeX)
  report/
    student4_report.tex      # this document
  presentation.pptx
  presentation_v2.pptx
```